

PYTHON CHO NGƯỜI MỚI – Đầy đủ 150 hàm, giải thích dễ hiểu

Giống bảng rút gọn nhưng đầy đủ hết. Đọc cho biết, không cần học thuộc – cần thì quay lại tra.

1. Hàm cơ bản

Hàm	Hiểu đơn giản là...	Ví dụ	Kết quả
<code>print()</code>	In chữ ra màn hình	<code>print("Chào")</code>	Chào
<code>type()</code>	Hỏi "cái này là loại gì?"	<code>type(5)</code>	<code>int</code>
<code>id()</code>	Hỏi "địa chỉ nhà" của biến trong bộ nhớ	<code>id(x)</code>	1 con số dài
<code>exit()</code>	Thoát chương trình	<code>exit()</code>	chương trình dừng
<code>input()</code>	Chờ người dùng gõ gì đó	<code>input("Tên: ")</code>	gõ gì trả về đó
<code>len()</code>	Đếm số lượng	<code>len("hi")</code>	2
<code>help()</code>	Hỏi "cái này dùng sao?"	<code>help(print)</code>	hiện hướng dẫn

2. Hàm số học

Hàm	Hiểu đơn giản là...	Ví dụ	Kết quả
<code>abs()</code>	Bỏ dấu âm	<code>abs(-5)</code>	5
<code>round()</code>	Làm tròn số	<code>round(3.678,1)</code>	3.7
<code>int()</code>	Ép thành số nguyên	<code>int("5")</code>	5
<code>float()</code>	Ép thành số thập phân	<code>float("5.5")</code>	5.5
<code>min()</code>	Tìm số nhỏ nhất	<code>min(3,7,1)</code>	1
<code>max()</code>	Tìm số lớn nhất	<code>max(3,7,1)</code>	7

Hàm	Hiểu đơn giản là...	Ví dụ	Kết quả
<code>pow()</code>	Tính lũy thừa	<code>pow(2,3)</code>	8
<code>divmod()</code>	Chia lấy cả thương lẫn dư luôn	<code>divmod(7,2)</code>	(3,1)

3. Hàm dãy (list, tuple...)

Hàm	Hiểu đơn giản là...	Ví dụ	Kết quả
<code>list()</code>	Tạo danh sách	<code>list("abc")</code>	<code>['a','b','c']</code>
<code>tuple()</code>	Tạo danh sách "khóa cứng" không sửa được	<code>tuple([1,2])</code>	<code>(1,2)</code>
<code>range()</code>	Tạo dãy số để lặp	<code>range(3)</code>	<code>0,1,2</code>
<code>sorted()</code>	Sắp xếp	<code>sorted([3,1,2])</code>	<code>[1,2,3]</code>
<code>reversed()</code>	Đảo ngược	<code>list(reversed([1,2,3]))</code>	<code>[3,2,1]</code>
<code>enumerate()</code>	Đánh số thứ tự kèm phần tử	<code>list(enumerate(['a','b']))</code>	<code>[(0,'a'), (1,'b')]</code>
<code>map()</code>	Áp dụng 1 phép cho từng phần tử	<code>list(map(str,[1,2]))</code>	<code>['1','2']</code>
<code>filter()</code>	Lọc ra phần tử thỏa điều kiện	<code>list(filter(lambda x:x>2,[1,2,3]))</code>	<code>[3]</code>
<code>slice()</code>	Cắt một đoạn trong dãy	<code>a[slice(1,3)]</code>	phần tử từ vị trí 1-2

4. Hàm xử lý chữ (string)

Hàm	Hiểu đơn giản là...	Ví dụ	Kết quả
<code>str()</code>	Ép thành chữ	<code>str(10)</code>	"10"
<code>chr()</code>	Số → ký tự	<code>chr(65)</code>	"A"
<code>format()</code>	Ghép chuỗi có định dạng	<code>"{}-{}".format(1,2)</code>	"1-2"
<code>find()</code>	Tìm vị trí (không thấy trả -1)	<code>"hello".find("l")</code>	2
<code>index()</code>	Tìm vị trí (không thấy thì báo lỗi)	<code>"hello".index("l")</code>	2
<code>replace()</code>	Thay chữ này bằng chữ khác	<code>"hi".replace("h","H")</code>	"Hi"

5. Hàm tập hợp / collection

Hàm	Hiểu đơn giản là...	Ví dụ	Kết quả
<code>frozenset()</code>	Tập hợp không trùng, không sửa được	<code>frozenset([1,2,2])</code>	{1,2}
<code>dict()</code>	Tạo kiểu tên-giá trị	<code>dict(a=1,b=2)</code>	{'a':1,'b':2}
<code>bool()</code>	Ép thành Đúng/Sai	<code>bool(0)</code>	False
<code>set()</code>	Tập hợp không trùng lặp	<code>set([1,1,2])</code>	{1,2}

6. Hàm lặp (iterable)

Hàm	Hiểu đơn giản là...	Ví dụ	Kết quả
<code>iter()</code>	Biến thành thứ "lặp được từng cái một"	<code>it = iter([1,2,3])</code>	sẵn sàng để <code>next()</code>
<code>next()</code>	Lấy phần tử kế tiếp	<code>next(it)</code>	1
<code>zip()</code>	Ghép 2 danh sách theo cặp	<code>list(zip([1,2], ['a','b']))</code>	[(1,'a'), (2,'b')]
<code>all()</code>	Đúng nếu TẤT CẢ đều đúng	<code>all([True,True])</code>	True
<code>any()</code>	Đúng nếu CÓ ÍT NHẤT 1 cái đúng	<code>any([False,True])</code>	True

7. Băm (hash)

Hàm	Hiểu đơn giản là...	Ví dụ	Kết quả
<code>hash()</code>	Tạo 1 mã số đại diện cho dữ liệu	<code>hash("abc")</code>	1 số cố định

8. Làm việc với file

Hàm	Hiểu đơn giản là...	Ví dụ	Kết quả
<code>open()</code>	Mở file ra dùng	<code>f = open("a.txt", "r")</code>	file đã mở
<code>readline()</code>	Đọc 1 dòng	<code>f.readline()</code>	dòng đầu tiên
<code>readlines()</code>	Đọc hết, trả về từng dòng trong 1 list	<code>f.readlines()</code>	<code>['dòng1\n', 'dòng2\n']</code>
<code>write()</code>	Ghi chữ vào file	<code>f.write("hi")</code>	file có thêm chữ "hi"
<code>writelines()</code>	Ghi nhiều dòng 1 lượt	<code>f.writelines(["a\n", "b\n"])</code>	file có 2 dòng
<code>close()</code>	Đóng file lại	<code>f.close()</code>	file đóng
<code>seek()</code>	Nhảy con trỏ đến vị trí	<code>f.seek(0)</code>	về đầu file

Hàm	Hiểu đơn giản là...	Ví dụ	Kết quả
	bất kỳ		
<code>tell()</code>	Hỏi con trỏ đang ở đâu	<code>f.tell()</code>	vị trí hiện tại
<code>remove()</code>	Xóa file	<code>os.remove("a.txt")</code>	file bị xóa

9. Hệ điều hành — `os`

Hàm	Hiểu đơn giản là...	Ví dụ	Kết quả
<code>import os</code>	Nạp bộ công cụ làm việc với máy tính	<code>import os</code>	sẵn sàng dùng
<code>os.getcwd()</code>	Hỏi "mình đang ở thư mục nào?"	<code>os.getcwd()</code>	đường dẫn hiện tại
<code>os.listdir()</code>	Liệt kê file trong thư mục	<code>os.listdir(".")</code>	danh sách tên file
<code>os.mkdir()</code>	Tạo thư mục mới	<code>os.mkdir("data")</code>	thư mục "data" được tạo
<code>os.rename()</code>	Đổi tên file	<code>os.rename("a.txt", "b.txt")</code>	file đổi tên
<code>os.path.exists()</code>	Hỏi "file/thư mục này có tồn tại không?"	<code>os.path.exists("a.txt")</code>	True / False

10. Ngày & giờ — datetime

Hàm	Hiểu đơn giản là...	Ví dụ	Kết quả
<code>import datetime</code>	Nạp bộ công cụ ngày giờ	<code>import datetime</code>	sẵn sàng dùng
<code>datetime.now()</code>	Lấy ngày giờ hiện tại	<code>datetime.datetime.now()</code>	vd 2026-08-09 22:00
<code>date.today()</code>	Lấy ngày hôm nay	<code>datetime.date.today()</code>	vd 2026-08-09
<code>strftime()</code>	Đổi ngày giờ thành chữ theo ý mình	<code>now.strftime("%d/%m/%Y")</code>	09/08/2026
<code>strptime()</code>	Đổi chữ thành ngày giờ	<code>strptime("2026-01-01", "%Y-%m-%d")</code>	đối tượng ngày
<code>timedelta()</code>	Cộng/trừ thời gian	<code>timedelta(days=7)</code>	"7 ngày" để cộng vào ngày khác

11. Ngẫu nhiên — random

Hàm	Hiểu đơn giản là...	Ví dụ	Kết quả
<code>import random</code>	Nạp bộ công cụ random	<code>import random</code>	sẵn sàng dùng
<code>random()</code>	Số thực ngẫu nhiên từ 0 đến 1	<code>random.random()</code>	vd 0.37
<code>randint()</code>	Số nguyên ngẫu nhiên trong khoảng	<code>random.randint(1,10)</code>	vd 7
<code>uniform()</code>	Số thực ngẫu nhiên trong khoảng	<code>random.uniform(1,10)</code>	vd 4.82
<code>choice()</code>	Bốc ngẫu nhiên 1 phần tử	<code>random.choice([1,2,3])</code>	vd 2
<code>shuffle()</code>	Xáo trộn thứ tự	<code>random.shuffle(my_list)</code>	list bị xáo trộn
<code>sample()</code>	Bốc ngẫu nhiên nhiều phần tử, không trùng	<code>random.sample([1,2,3],2)</code>	vd [3,1]

12. Tiện ích cho hàm

Hàm	Hiểu đơn giản là...	Ví dụ	Kết quả
<code>def</code>	Đặt tên cho 1 đoạn code để xài lại	<code>def cong(a,b): return a+b</code>	gọi <code>cong(2,3)</code> → 5
<code>isinstance()</code>	Hỏi "cái này có phải kiểu X không?"	<code>isinstance(5,int)</code>	True
<code>issubclass()</code>	Hỏi "lớp này có kế thừa từ lớp kia không?"	<code>issubclass(bool,int)</code>	True
<code>hasattr()</code>	Hỏi "đối tượng này có thuộc tính này không?"	<code>hasattr(obj,"name")</code>	True / False
<code>globals()</code>	Xem tất cả biến toàn cục đang có	<code>globals()</code>	danh sách biến
<code>vars()</code>	Xem thuộc tính của 1 đối tượng	<code>vars(obj)</code>	các thuộc tính

13. Hằng số dựng sẵn

Tên	Hiểu đơn giản là...
<code>True</code> / <code>False</code>	Đúng / Sai
<code>None</code>	"Không có gì cả"
<code>NotImplemented</code>	"Chưa hỗ trợ phép toán này"
<code>Ellipsis (...)</code>	Dấu ba chấm giữ chỗ, ít dùng lúc mới học

14. Hàm nâng cao

Hàm	Hiểu đơn giản là...	Ví dụ	Kết quả
<code>callable()</code>	Hỏi "cái này gọi được như hàm không?"	<code>callable(print)</code>	True
<code>compile()</code>	Biến chuỗi code thành thứ chạy được	<code>compile("1+1", "<s>", "eval")</code>	code đã biên dịch

Hàm	Hiểu đơn giản là...	Ví dụ	Kết quả
<code>exec()</code>	Chạy 1 đoạn code viết dưới dạng chữ	<code>exec("x=5")</code>	biến <code>x</code> được tạo
<code>eval()</code>	Tính luôn kết quả của 1 biểu thức viết dưới dạng chữ	<code>eval("2+3")</code>	5
<code>lambda</code>	Hàm siêu ngắn, viết trong 1 dòng	<code>f = lambda x: x*2</code>	gọi <code>f(3)</code> → 6

15. Xử lý lỗi

Từ khóa	Hiểu đơn giản là...	Ví dụ	Kết quả
<code>try</code>	"Thử làm cái này xem"	<code>try: 1/0</code>	bắt đầu thử
<code>except</code>	"Nếu lỗi thì làm cái này thay vì crash"	<code>except: print("lỗi")</code>	in lỗi
<code>raise</code>	Tự mình tạo ra 1 lỗi	<code>raise ValueError("sai")</code>	chương trình báo lỗi
<code>finally</code>	"Dù đúng dù sai vẫn phải làm cái này"	<code>finally: f.close()</code>	luôn đóng file
<code>assert</code>	"Nếu điều này sai thì báo lỗi ngay"	<code>assert x > 0</code>	lỗi nếu $x \leq 0$

16. Gỡ lỗi (debug)

Hàm	Hiểu đơn giản là...	Ví dụ	Kết quả
<code>breakpoint()</code>	Tạm dừng chương trình để soi lỗi	<code>breakpoint()</code>	chương trình dừng lại chờ
<code>logging.debug()</code>	Ghi chú chi tiết để debug	<code>logging.debug("chi tiết")</code>	ghi vào log
<code>logging.info()</code>	Ghi chú thông tin thường	<code>logging.info("đang chạy")</code>	ghi vào log
<code>logging.error()</code>	Ghi chú khi có lỗi	<code>logging.error("có lỗi")</code>	ghi vào log

17. Module

Hàm	Hiểu đơn giản là...	Ví dụ	Kết quả
<code>import</code>	Nạp thêm bộ công cụ có sẵn	<code>import math</code>	dùng được các hàm trong <code>math</code>
<code>dir()</code>	Xem đối tượng này có những gì bên trong	<code>dir(str)</code>	danh sách hàm của chuỗi
<code>__import__()</code>	Nạp module bằng cách gõ tên dạng chữ	<code>__import__("os")</code>	module <code>os</code> được nạp
<code>reload()</code>	Nạp lại module sau khi sửa code	<code>importlib.reload(math)</code>	module được cập nhật

18. Bộ nhớ (memory)

Hàm	Hiểu đơn giản là...	Ví dụ	Kết quả
<code>memoryview()</code>	Nhìn vào dữ liệu thô mà không copy ra	<code>memoryview(b"abc")</code>	view của dữ liệu
<code>bytearray()</code>	Tạo mảng byte có thể sửa được	<code>bytearray(5)</code>	<code>bytearray(b'\x00\x00\x00\x00\x00')</code>
<code>bytes()</code>	Tạo dữ liệu byte, không sửa được	<code>bytes("abc", "utf-8")</code>	<code>b'abc'</code>

Hàm	Hiểu đơn giản là...	Ví dụ	Kết quả
<code>array()</code>	Tạo mảng số cùng 1 kiểu, tiết kiệm bộ nhớ	<code>array.array('i', [1,2,3])</code>	mảng số nguyên

Ghi nhớ

Nhóm **1, 2, 3, 4, 12, 15** là quan trọng nhất lúc mới học — mấy nhóm còn lại đọc lướt qua, cần thì quay lại tra là được.